

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Constraint Programming

Tractable Structure and Symmetry

Week 5

Tractable Structure and Symmetry

Assoc. Prof. Tolga Berber

KTU Faculty of Science, Department of Computer Science · https://avesis.ktu.edu.tr/tberber

Learning phases: 1 • Pre-instruction 2 • Content 3 • Participation 4 • Assessment 5 • Follow-through

1 • Pre-instruction Why It Matters?

Weeks 3–4 taught you how to *survive* a search tree that might, in the worst case, force exponentially many backtracks.

- This week asks a different question: independent of *how* you search, does the problem's own shape make it easy or hard?
- Some constraint graphs – recognizable before you write a single line of search code – guarantee a search that never backtracks at all
- Other instances are hard by pure repetition: many of their "different" solutions are the same underlying answer, relabeled

This Week

◎ Learning Objectives

By the end of this week, you will be able to:

1. Recognize a tree-structured CSP, apply directional arc consistency (DAC), and explain why forward assignment is then backtrack-free
2. Explain cutset conditioning and tree decomposition, and compute the induced width of a small constraint graph for an ordering
3. Explain what makes a CSP tractable versus NP-hard, and relate bounded treewidth to Week 4's search engineering
4. Identify variable/value symmetry and apply lexicographic-ordering symmetry-breaking constraints in CP-SAT
5. Build and run CP-SAT models with and without symmetry breaking, and read the solver's own search statistics as evidence

SECTION 1

Unit 1 — Tree-Structured CSPs and Directional Arc Consistency

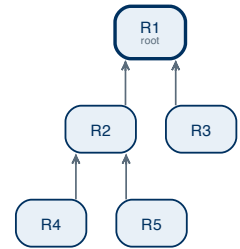
2 • Content A Constraint Graph With No Cycles

- Recall (Week 1): a constraint graph has one node per variable, one edge per binary constraint
- Week 1's Australia map has a triangle WA–NT–SA – not a tree

Definition: Tree-structured CSP

A CSP whose constraint graph is a tree: connected, with no cycles.

A Genuine Tree-Shaped Map



A tree-structured constraint graph — contrast with Week 1's Australia map, which has a triangle (WA–NT–SA) and is not a tree.

R1 is the root; R2, R3 are its children; R4, R5 are R2's children – 5 nodes, 4 edges, no cycle.

2 • Content Ordering the Variables, Then Filtering

Definition: Tree-consistent ordering

A variable ordering in which every variable after the first has exactly one earlier neighbour in the constraint graph – its parent.

Definition: Directional arc consistency (DAC)

Process the variables from last to second; for each one, remove from its parent's domain every value with no compatible value left in the child's domain – one child at a time, in reverse order.

2 • Content Setting Up the Tree Map, in Code

```python
REGIONS = ["R1", "R2", "R3", "R4", "R5"]
EDGES = [("R1", "R2"), ("R1", "R3"), ("R2", "R4"), ("R2", "R5")]

def build_domains() -> dict[str, list[int]]:
    """One 3-colour domain (0, 1, 2) per region, before any DAC
       filtering."""
    return {region: [0, 1, 2] for region in REGIONS}
```

One 3-colour domain per region, before any filtering – the same adjacency the figure just showed, as plain Python structures.

2 • Content DAC, in Code

```python
def apply_dac(domains: dict[str, list[int]]) -> None:
    """Directional arc consistency, processed in reverse tree-consistent
    order (last region to second): for each region, remove from its
    parent's domain every value with no compatible value left in the
    region's own (current) domain.
    """
    for child in reversed(ORDER[1:]):
        parent = PARENT[child]
        domains[parent] = [
            value
            for value in domains[parent]
            if any(is_compatible(value, other) for other in domains[child])
        ]
        print(f"Revise {parent} against {child}: domains[{parent}] = {domains[
            parent]}")
```

2 • Content The DAC Trace

Expected Output

```
Revise R2 against R5: domains[R2] = [0, 1, 2]
Revise R2 against R4: domains[R2] = [0, 1, 2]
Revise R1 against R3: domains[R1] = [0, 1, 2]
Revise R1 against R2: domains[R1] = [0, 1, 2]
```

Every domain here has 3 colours and 3-colouring a tree never removes a value – DAC's real payoff shows up in the forward pass next, not in a domain shrinking on this particular instance.

Freuder's Tree Theorem

Stated, not proved

If a tree-structured CSP is DAC with respect to a tree-consistent ordering, assigning variables forward, root to leaves, choosing any remaining domain value at each step, never needs to backtrack.

Analogy

A course prerequisite chain with no cycles, where every course depends on exactly one earlier course: walking the tree once, root to leaves, you never need to undo an earlier choice – the *shape* of the chain guarantees it, not luck.

2 • Content Forward Assignment: Zero Backtracks

- Root to leaves, pick the first domain value still compatible with the already-assigned parent – DAC guarantees one always exists
- A plain CP-SAT solve of the same instance cross-checks the result

Expected Output

```
Assign R1 = Red
Assign R2 = Green
Assign R3 = Green
Assign R4 = Red
Assign R5 = Red
Backtracks needed: 0
CP-SAT cross-check status: OPTIMAL, valid coloring: True
```

3 • Participation Quick Check: Star Graph

A 4-node constraint graph has edges A–B, B–C, B–D (a star centred on B), no other edges. Is it tree-structured? Give a tree-consistent ordering, and name which variable DAC revises last.

Yes – a star is a tree. One ordering: B, A, C, D. Reverse-order processing visits D first, then C, then A, each time revising B's domain – B itself, the root, is never revised.

SECTION 2

Unit 2 — Cutset Conditioning, Tree Decomposition and Treewidth

2 • Content When the Graph Is Almost a Tree

Definition: Cutset conditioning

Remove a small set of variables (the cycle *cutset*) so that the rest is a tree; fix a value for each cutset variable in turn, then solve the remaining tree-structured CSP backtrack-free for each combination.

Analogy

A lab with one shared microscope: if several independent group schedules only ever conflict over one resource, fixing who uses it when turns one tangled schedule into several small, easy, independent ones.

2 • Content Grouping Variables Into Overlapping Bags

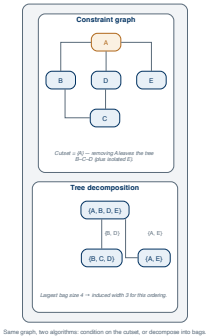
Definition: Tree decomposition

A tree of overlapping variable “bags” such that every constraint’s variables share a bag, and any variable shared by two bags appears in every bag between them – generalizes Unit 1’s algorithm to any graph.

Definition: Induced width / treewidth

Given an elimination ordering, the *induced width* is the largest number of still-active neighbours any variable acquires as earlier variables are eliminated. *Treewidth* is the minimum induced width over all orderings.

Cutset and Tree Decomposition, Side by Side



Removing A leaves the tree B–C–D; largest bag size 4 $\rightarrow$ induced width 3.

2 • Content Computing Induced Width, in Code

```python
def induced_width(adjacency: Adjacency, order: list[str]) -> int:
    """Simulate variable elimination in `order`: at each step, connect the
    eliminated variable's still-active neighbours into a clique (fill-in
    edges), and track the largest neighbour count seen at elimination time.
    """
    graph = {node: set(neighbours) for node, neighbours in adjacency.items()}
    width = 0
    for node in order:
        neighbours = graph[node]
        width = max(width, len(neighbours))
        for neighbour in neighbours:
            graph[neighbour].update(neighbours - {neighbour})
        for neighbour in neighbours:
            graph[neighbour].discard(node)
        del graph[node]
    return width
```

2 • Content Induced Width on Both Worked Graphs

Expected Output

```
Elimination order: ['R3', 'R4', 'R5', 'R2', 'R1']
Induced width: 1
Cutset variable eliminated first: A
Elimination order: ['A', 'B', 'D', 'E', 'C']
Induced width: 3
```

A tree's treewidth is always 1; eliminating cutset variable A first connects its three remaining neighbours into a clique, giving width 3.

3 • Participation Quick Check: 4-Cycle Plus One Edge

A 5-node graph has edges A–B, B–C, C–D, D–A (a 4-cycle) plus A–E. Name a variable whose removal leaves a tree, and give the induced width of eliminating it first.

Removing A breaks the cycle into path B–C–D (E is isolated) – a tree. Eliminating A first connects B, D, E into a clique (3 fill-in edges), giving induced width 3 for that ordering.

SECTION 3

Unit 3 — Complexity: What Makes a CSP Hard

2 • Content CSP Is NP-Hard in General

- Recall (Week 1): Boolean satisfiability (SAT) is the special case of a CSP with Boolean domains; SAT is NP-complete
- So CSP in general is NP-hard – no known algorithm solves every instance in time polynomial in the number of variables

Definition: Tractable problem class

A class of problems solvable in time polynomial in the input size – e.g., CSPs of fixed, bounded treewidth.

2 • Content Bounded Treewidth: A Genuine Tractable Island

- For a fixed treewidth k, Unit 2's tree-decomposition algorithm solves a CSP in time polynomial in the number of variables (though exponential in k)
- Tree-structured CSPs (Unit 1, treewidth 1) are the simplest case of this same result, not a separate phenomenon
- Tightness and density also affect practical difficulty – but treewidth is the one lever with a matching worst-case guarantee

Analogy: Org-Chart Budgets vs Fully Cross-Referenced Ones

Analogy

Reconciling a fully cross-referenced spreadsheet of department budgets versus an org-chart-shaped one: a tangled graph can blow up combinatorially as departments are added; a tree-like graph stays manageable no matter how many there are.

2 • Content Structure and Search Engineering Are Complementary

- Week 4's restarts, nogood learning and local search make many *practical* instances fast – without lowering CSP's *worst-case* complexity
- An instance with unbounded treewidth and no exploitable symmetry can still force exponential search time, however well-engineered
- Structure explains provable easy cases; search engineering explains good typical-case behaviour on the rest

3 • Participation Quick Check: Fixed vs Growing Treewidth

Instance X's constraint graph is a tree; instance Y's has treewidth roughly one third of the number of variables. Which does Unit 2's algorithm guarantee to solve in polynomial time?

X only – treewidth 1 is fixed and small, so the exponential term stays fixed. Y's treewidth grows *with* instance size, so the guarantee (which only holds for *fixed* treewidth) does not apply.

SECTION 4

Unit 4 — Symmetry and Dominance

2 • Content Two Kinds of Symmetry

Definition: Symmetry

A transformation of a solution – relabeling variables, values, or both – that always produces another valid solution to the same instance.

- **Value symmetry**: swapping “colour 1” and “colour 2” everywhere in a map colouring turns any solution into another
- **Variable symmetry**: relabeling “week 2” and “week 3” throughout a social-golfer schedule does the same

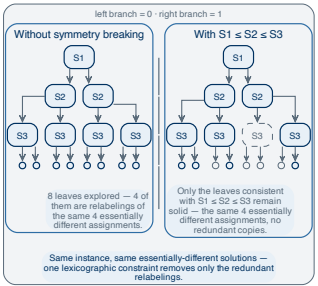
2 • Content Why Symmetry Hurts Search

Analogy

Four interchangeable teaching-assistant slots for the same lab session: a search that does not know they are identical tries all 24 relabelings of the exact same real assignment.

An unmodified search rediscovers every symmetric copy of the same underlying solution (or the same failure) separately – multiplying the search space without adding anything genuinely new to find.

Why an Unbroken Search Wastes Effort



Three interchangeable Boolean variables: 8 leaves in the full tree, but only 4 essentially different assignments – the rest are relabelings of those same 4.

2 • Content Symmetry-Breaking Constraints

Definition: Symmetry-breaking constraint

An added constraint that keeps at least one representative of every symmetry class of solutions while removing the redundant, symmetric copies – it must never remove an entire class.

Definition: Lexicographic ordering constraint

Force a sequence of interchangeable objects into one canonical order by requiring each one to be lexicographically less-than-or-equal to the next.

Analogy: An Alphabetized Reading List

Analogy

Keeping a reading list alphabetized: many orderings describe the same underlying list, but insisting on alphabetical order fixes exactly one canonical ordering – a lexicographic constraint does the same to an otherwise-interchangeable group of model variables.

2 • Content Breaking Symmetry in the Social Golfer Model

```python
def add_symmetry_breaking(model: cp_model.CpModel, play) -> None:
    """Week 1 fixed to a canonical grouping; groups within every other
    week ordered by smallest golfer id; weeks after week 1 lexicographically
    ordered against each other.
    """
    for golfer in range(NUM_GOLFERS):
        model.Add(play[0, golfer, golfer // GROUP_SIZE] == 1)
    for week in range(1, NUM_WEEKS):
        model.Add(play[week, 0, 0] == 1)
    for week in range(1, NUM_WEEKS - 1):
        left = [_group_of(play, week, golfer) for golfer in range(NUM_GOLFERS)
            ]
        right = [_group_of(play, week + 1, golfer) for golfer in range(
            NUM_GOLFERS)]
        _add_lex_le(model, left, right)
```

2 • Content Why Hand-Rolled Lex, Not Built-In

Common Pitfall

This constraint model has no built-in lexicographic constraint here – `ortools==9.14.6206`'s `CpModel` exposes no `*Lex*` method, so the ordering is hand-rolled: `left[i] <= right[i]` only while every earlier position is already equal.

Week 1 is fixed to a canonical grouping; groups within every other week are ordered by smallest golfer id; weeks after week 1 are lexicographically ordered against each other.

2 • Content Does It Cost Anything? Golfer Search Statistics

Expected Output

```
Without symmetry breaking:
NumBranches: 2352   NumConflicts: 147   WallTime: 0.011725

With symmetry breaking:
NumBranches: 1454   NumConflicts: 25    WallTime: 0.006005
```

Same 9-golfer, 4-week schedule, same solver seed – symmetry breaking cuts branches and conflicts roughly in half without changing feasibility.

2 • Content The Same Recipe on BIBD's Column Symmetry

```python
def add_symmetry_breaking(model: cp_model.CpModel, x) -> None:
    """Adjacent block-columns forced into non-decreasing lexicographic
    order, one column-relabeling symmetry-breaking constraint per pair
        .
    """
    for block in range(NUM_BLOCKS - 1):
        left = [x[point, block] for point in range(NUM_POINTS)]
        right = [x[point, block + 1] for point in range(NUM_POINTS)]
        _add_lex_le(model, left, right)
```

Adjacent block-columns of the Fano-plane design $(v, b, r, k, \lambda) = (7, 7, 3, 3, 1)$ forced into non-decreasing lexicographic order – one column-relabeling symmetry removed per adjacent pair.

2 • Content BIBD Search Statistics

Expected Output

```
Without symmetry breaking:
NumBranches: 1185   NumConflicts: 0   WallTime: 0.005635

With symmetry breaking:
NumBranches: 599    NumConflicts: 0   WallTime: 0.005082
```

Both variants find a valid design – symmetry breaking again roughly halves the branches explored.

Beyond Symmetry: Dominance Detection

- Symmetry breaking requires an *exact* relabeling between two solutions
- **Dominance detection** generalizes this: prune a partial assignment whenever it is provably no better than one already explored, even without an exact symmetry
- Named here as the natural next step – not built this week

3 • Participation Quick Check: Interchangeable Review Slots

Three interchangeable slots Slot1, Slot2, Slot3 must each get one of reviewers X, Y, Z. Write one lexicographic-style constraint removing the slot-relabeling symmetry.

Fix Slot1's reviewer to the alphabetically-earliest reviewer appearing anywhere ($\texttt{Slot1} \leq \texttt{Slot2}$, $\texttt{Slot1} \leq \texttt{Slot3}$ under $X<Y<Z$) – one representative per relabeling class, every genuinely different assignment kept.

SECTION 5

Summary

Summary

Summary

- A tree-structured graph, made DAC with respect to a tree-consistent ordering, guarantees backtrack-free forward assignment
- Cutset conditioning and tree decomposition both reduce a non-tree graph's cost to one number: the treewidth
- Bounded treewidth is a genuine tractable island inside a generally NP-hard problem class – Week 4's search engineering helps the typical case without changing this worst case
- Symmetry-breaking constraints, most often a lexicographic ordering, remove redundant relabelings without losing any essentially different solution

4 • Assessment Check Yourself

- One Check Yourself question closes each unit in this week's chapter – answers are collected at the chapter's end
- A path graph's tree-consistent ordering, a 6-cycle's cutset and induced width, a near-complete graph's treewidth guarantee, a lexicographic constraint for four interchangeable shift variables

5 • Follow-through Transfer — Where Else?

- Sketching a new problem's constraint graph and asking "close to a tree, or hopelessly tangled?" is a five-minute check before you write any search code
- Column/row symmetry of a 0/1 design matrix reappears in timetabling's day-relabeling symmetry (Week 12) and routing's depot-relabeling symmetry (Week 14)
- Next week keeps the same modeling discipline – viewpoints, auxiliary variables, implied constraints, and now symmetry-breaking constraints – and expresses it once in a solver-independent language

In the Lab

This week's lab sheet

You run the DAC trace and verify the zero-backtrack assignment yourself, compute induced width on both worked graphs, sketch and classify a new constraint graph's tractability, then run both social-golfer and BIBD variants and compare the real search statistics.

- Capstone step: add one further lexicographic constraint (BIBD's rows, in addition to its columns) and confirm the design still holds
- Next week: the same modeling discipline, expressed once in MiniZinc and run unchanged on two different solvers